

A var args is internally array but unlike array we cannot use as a local variable i.e. var args we can use only for parameter variable.
Example :
void main(String ...args) //Var args is **valid** here because using as a parameter
{
 int ...arr = {10,20,30}; //Var args is **Invalid** here because using as a parameter
}
* Whenever we use var args as a method OR constructor **parameter** then we have the flexibility to pass **direct values OR array variable to the var args because var args can accept both i.e direct values OR array variable**.
* Var args must be **only one and last parameter** otherwise we will get a compilation error.

WAP that describes var args can accept 0 to n of arguments :

```
VarArgs1.java
-----
void main(String ...args)
{
    accept();
    accept(1);
    accept(5,10);
    accept(5,10,15);
    accept(5,10,15,20);
    accept(5,10,15,20,25);
}

void accept(int ...x)
{
    IO.println("Executed");
}
-----
WAP to show var args can accept all different types of values.
```

```
VarArgs2.java
-----
void main()
{
    acceptHetro(12, 23.90, 'A', true, "Java");
}

void acceptHetro(Object ...values)
{
    for(Object value : values)
    {
        IO.println(value);
    }
}
-----
WAP to show var args can add parameter variable values.
```

```
VarArgsDemo3.java
-----
void main()
{
    addParameter(10,20);
    addParameter(100,200,300);
    addParameter(1000,2000,3000,4000);
}

void addParameter(int ...values)
{
    int sum = 0;

    for(int value : values)
    {
        sum = sum + value;
    }
    IO.println("Sum of the parameter is :"+sum);
}
-----
WAP to show the real life use of var args for any shopping application to calculate the cart price.
```

```
VarArgs4.java
-----
void main()
{
    addCartItemPrice(10,20);
    addCartItemPrice(100,200,300);
    addCartItemPrice(1000,2000,3000,4000);
}

public void addCartItemPrice(double ...prices)
{
    double billAmount = 0;
    for(double price : prices)
    {
        billAmount = billAmount + price;
    }

    IO.println("Dear customer, Your final bill is :"+billAmount);
}
-----
WAP to show that var args can accept direct values OR array variable values.
```

```
VarArgs5.java
-----
void main()
{
    IO.println("Passing Values directly...");
    accept(10,20,30,40,50);
    IO.println("Paasing Array Variable....");
    int []values = {60,70,80,90,100};
    accept(values);
}

void accept(int ...arr)
{
    for(int x : arr)
    {
        IO.print(x+" ");
    }
}
-----
WAP to show that var args must be only one and last parameter.
```

```
VarArgs6.java
-----
void main()
{
    accept(10,20,30,40,50);
}

/*
void accept(int ...x, int ...y) //Invalid
{
}

void accept(int ...x, int y) //Invalid
{
}
*/

void accept(int x, int ...y)
{
    IO.println("x value is :"+x);

    for(int z : y)
    {
        IO.println(z);
    }
}
-----
```

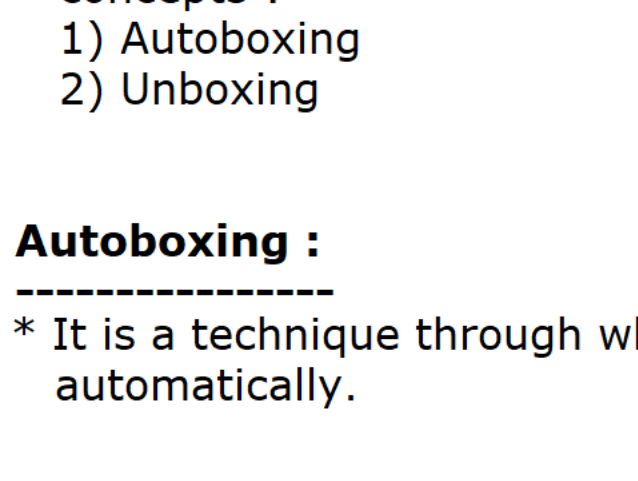
Autoboxing and Unboxing :

- * Java is not a pure object oriented language because it is accepting primitive data types.
- * Only objects are moving in the network but not the primitive data types.

Example :

```
-----
class Student
{
    int roll;
    String name;
    double height;
    int age;
}

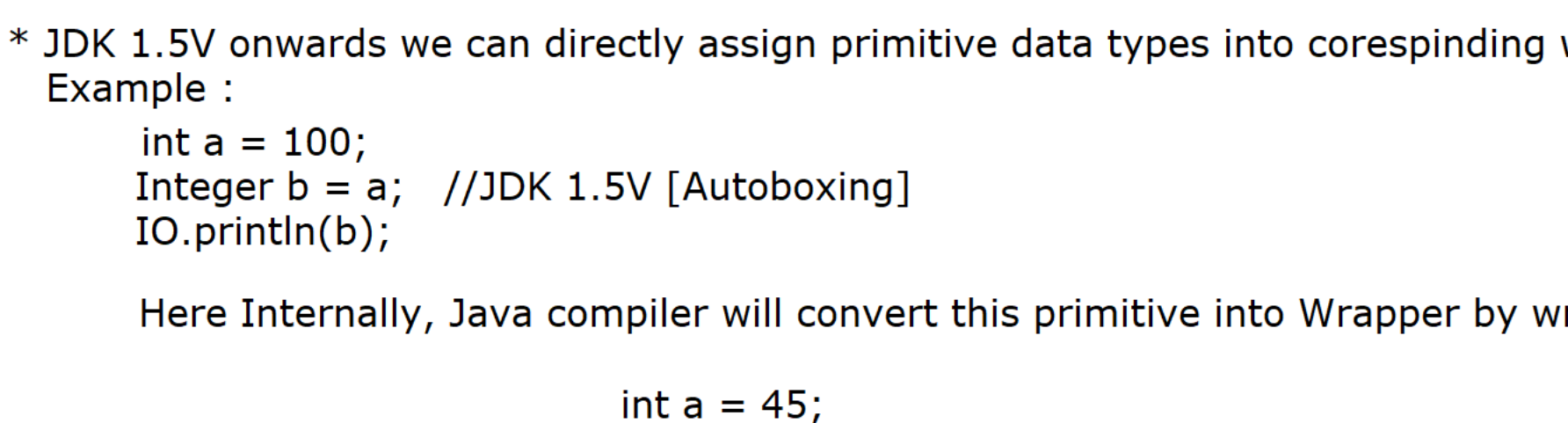
Student raj = new Student();
```



- * In order to convert the primitive data types into corresponding object, Java has provided Wrapper classes concept.
- * In order to convert from primitive to wrapper and vice versa, From JDK 1.5V onwards java has provided two concepts :
1) Autoboxing
2) Unboxing

Autoboxing :

* It is a technique through which we can convert the primitive data types into corresponding Wrapper object automatically.



* Till JDK 1.4V, A developer was responsible to convert the primitive data types into corresponding wrapper Object by writing valueOf() static method.

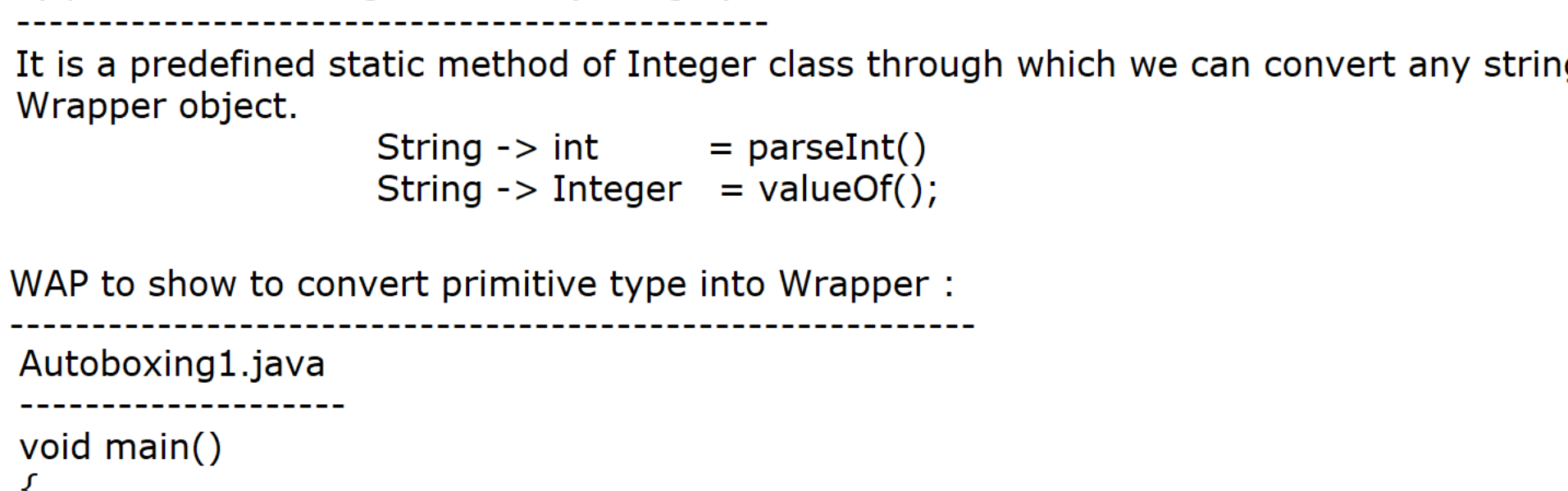
```
Example :
-----
int x = 90;
Integer obj = Integer.valueOf(x); //JDK 1.4V
IO.println(obj);
```

* JDK 1.5V onwards we can directly assign primitive data types into coresponding wrapper object.

Example :

```
int a = 100;
Integer b = a; //JDK 1.5V [Autoboxing]
IO.println(b);
```

Here Internally, Java compiler will convert this primitive into Wrapper by writing the follwing code.



* The following diagram explains primitive to Wrapper conversion :

Primary Data type	Wrapper Object
byte	: Byte
short	: Short
int	: Integer
long	: Long
float	: Float
double	: Double
char	: Character
boolean	: Boolean

Methods :

1) public static Integer valueOf(int x) :

* It is a predefined static method of Integer class through which we can convert primitive int type into Integer Object.
Note : All the Wrapper classes (8 Wrapper classes) have valueOf() method to convert the primitive into corresponding Wrapper.

2) public static Integer valueOf(String x) :

It is a predefined static method of Integer class through which we can convert any string type value into Integer Wrapper object.

```
String -> int = parseInt()
String -> Integer = valueOf();
```

WAP to show to convert primitive type into Wrapper :

```
Autoboxing1.java
-----
void main()
{
    int x = 90;
    Integer obj = Integer.valueOf(x); //JDK 1.4V
    IO.println(obj);

    int a = 100;
    Integer b = a; //JDK 1.5V [Autoboxing]
    IO.println(b);
}
-----
```

WAP to convert the String into Wrapper type :

```
Autoboxing2.java
-----
void main()
{
    String str = "123";
    Integer obj = Integer.valueOf(str);
    IO.println(obj + 2);
}
-----
```